and making that method, and only that method, protected. Access declared this way does not propagate down the file, it only applies to the method that the access modifier directly precedes.

We prefer this last form, because it's much more explicit about the access level of each method. That said, the first form is older, and currently more common in code you are likely to see.

It's time for some examples. Perhaps we're modeling an accounting system where every debit has a corresponding credit. Because we want to ensure that no one can break this rule, we'll make the methods that do the debits and credits private, and we'll define our external interface in terms of transactions.

```
class Account
  attr_accessor :balance

  def initialize(balance)
    @balance = balance
  end
end

class Transaction
  def initialize(account_a, account_b)
    @account_a = account_a
    @account_b = account_b
  end

  def transfer(amount)
    debit(@account_a, amount)
    credit(@account_b, amount)
  end

  private def debit(account, amount)
    account.balance -= amount
  end

  private def credit(account, amount)
    account.balance += amount
  end
end

savings = Account.new(100)
checking = Account.new(200)

transaction = Transaction.new(checking, savings)
transaction.transfer(50)
```

Protected access is used when objects need to access the internal state of other objects of the same class. For example, we may want to allow two individual Account objects to compare their balances directly but to hide those balances from the rest of the world (perhaps because we present them in a different form):

```
class Account
  protected attr_reader :balance  # accessor method 'balance' but make it protected

  def greater_balance_than?(other)
    @balance > other.balance
  end
end
```